

Group 6: Architecture Design

Sidharth Krishnaswamy, Nishit Oberoi, Edmond Pan, Kyle Kawasuna Saiki, Lu Maw Tun, Katia Yarkov

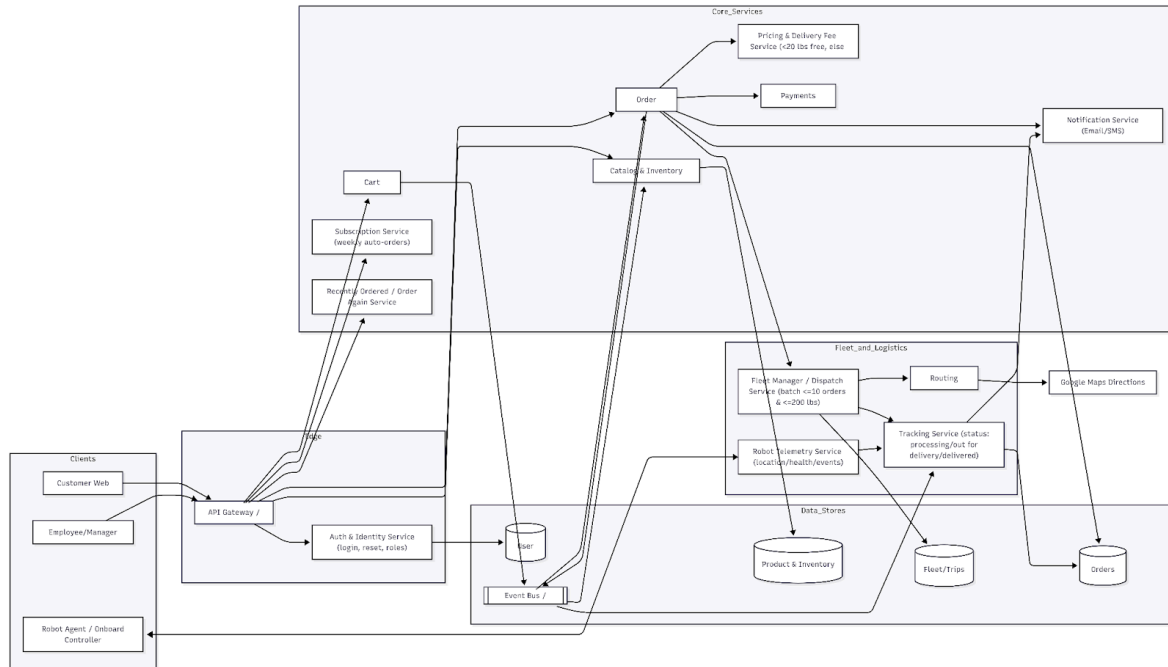
System Overview:

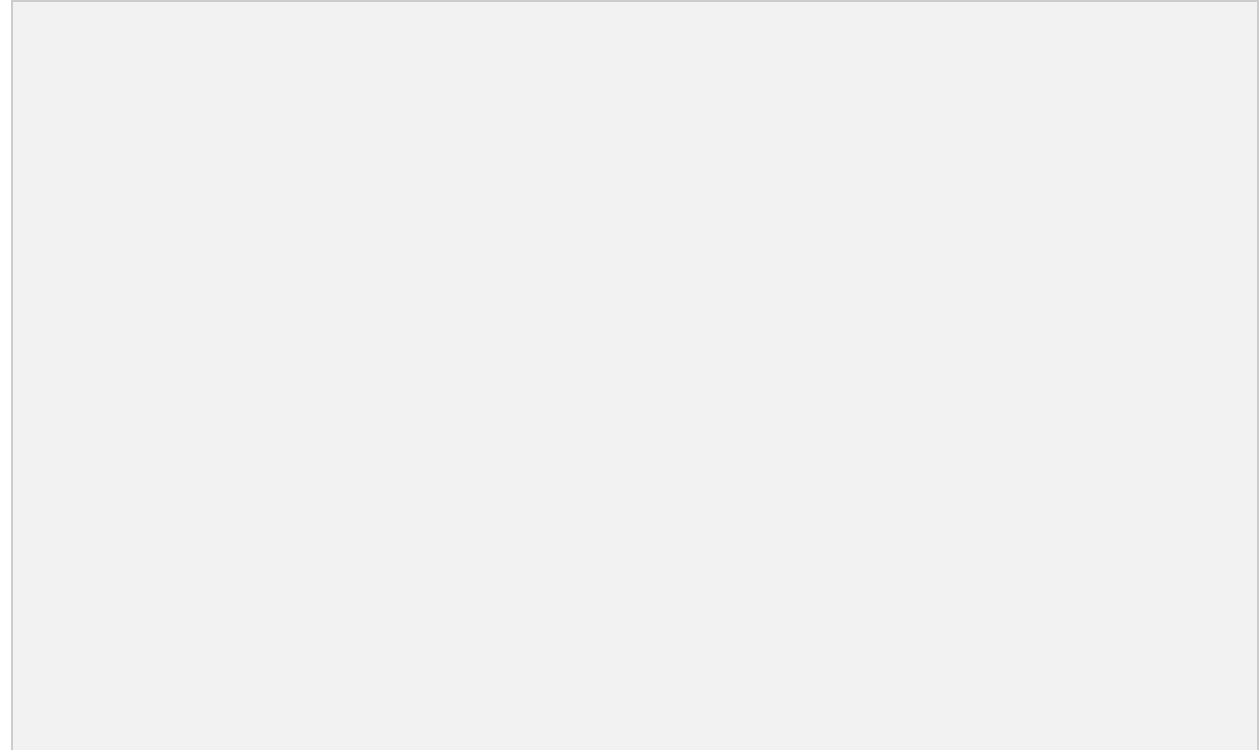
The OFS (Organic Food Service) system is a full-stack web application designed to support online grocery ordering and autonomous robotic delivery within Downtown San Jose. The system allows customers to browse organic grocery products, manage their own shopping cart, fulfill secure online payments, and schedule home deliveries. There is also an administrative dashboard that supports the needs of OFS's employees and managers. They are able to view product inventory, monitor orders, and view financial metrics.

This system will consist of a three-tier architecture with a frontend client layer, backend application layer, and relational database layer. We will implement the frontend layer with React to develop the user interface and possible user interactions. The backend will be developed using FastAPI, a Python based web framework, to implement RESTful APIs. This layer will handle user authentication, order processing, product filtering and searching, delivery fee calculation based on weight, inventory updates, and delivery scheduling. The relational database layer will be implemented with MySQL and store users, products, orders, and delivery information.

The system additionally integrates Google Maps API as an external service to handle traffic conditions and routing optimization for the delivery functionality. The backend will communicate with the Google Maps API to retrieve routing data and potential travel times. This integration will allow efficient scheduling of the robotic delivery trips while maintaining a separation between the system's internal components and third-party services.

Component Architecture Diagram:





Frontend Components:

1. Authentication

Description:

Provides user interface for login, registration, and logout functionality. Manages authentication tokens received from the backend and attaches them to API requests.

Functional Specifications:

- Collect User credentials
- Send login request to backend API
- Store authentication token securely
- Attach token to protected API requests
- Clear token upon logout

2. Product Browsing

Description:

Enables customers to discover available grocery items through categorized views, search functionality, and dynamic filtering options. It retrieves product data from the backend API and displays it to the user in an organized interface. The component ensures that the

users can efficiently locate products based on pricing, weight, dietary tags, and availability.

Function Specifications:

- Retrieve product data from backend API
- Display product attributes:
 - Name, price, Weight, organic certification label, stock availability
- Support key-word search functionality
- Enable filtering by:
 - Price range, weight range, dietary or organic tags
- Transmit user-selected filter criteria to backend layer and receives corresponding results
- Indicate out-of-stock products visually
- Update product display in response to user interaction, without page refreshes

3. Product Detail

Description:

Displays detailed information about a selected product. It allows users to review product specifications and add the item to their shopping cart.

Function Specifications:

- Displays product name, description, price, weight, dietary/organic labels, and current inventory availability
- Displays product image
- Allows users to choose a quantity before adding item to their cart
- Disable “Add to Cart” functionality if item is out of stock

4. Shopping Cart

Description: Provides summary view of the products selected by the user. Allows the user to review the items they intend to purchase, see the total cost and total weight, and adjust the quantity of each item before proceeding to checkout.

Function Specifications:

- Display all products currently added
- Calculates and displays the total price and total weight of all items in cart
- Allows users to increase or decrease the quantity of an item
- Allows users to remove items from cart
- Updates the total price and total weight when quantities are adjusted
- Prevents adding items to cart if chosen quantity is over the amount in stock

5. Checkout

Description: Manages the final stages of the transaction lifecycle. It guides the user through address confirmation, delivery scheduling, and secure payment processing. This component interacts with the backend to calculate final costs (including dynamic delivery fees based on weight) and converts the contents of the Shopping Cart into a confirmed Order.

Function Specifications:

- Displays all items in the shopping cart, including quantities, individual prices, total price, and total weight
- Calculates and displays delivery fees, if applicable, based on total order weight
- Allows users to select delivery date and time window
- Provides secure environment for users to enter debit or credit card information
- Validates payment information and processes the transaction securely through a backend payment service
- Sends confirmed order to the backend service for processing
 - Backend will validate stock availability before finalizing order
- Sends user a confirmation message once the order is successfully processed
- Updates the shopping cart to empty after complete checkout

6. Order Tracking

Description:

Provides customers with real-time visibility into the status of their active orders. It serves as the primary interface for monitoring the autonomous delivery process, integrating data from the backend Tracking Service and Robot Telemetry Service to visualize the robot's journey from the fulfillment center to the customer's doorstep.

Functional Specifications:

- Real-Time Status Display:
 - Retrieve and display the current order status from the backend API.
 - Visualized states include: Processing, Out for Delivery, and Delivered.
- Live Map Visualization:
 - Render an interactive map using the Google Maps API.
 - Plot the delivery robot's real-time geolocation coordinates (latitude/longitude) received from the Robot Telemetry Service.
 - Display the route path from the store to the delivery address.
- ETA Calculation:
 - Display a dynamic Estimated Time of Arrival (ETA) that updates based on the robot's current speed and traffic conditions retrieved from Google Maps.
- Auto-Refresh Mechanism:

- Poll the backend API at set intervals (e.g., every 5–10 seconds) to update the robot's location marker without requiring a full page reload.
- Arrival Notification:
 - Trigger a visual alert on the UI when the status changes to "Delivered" or when the robot arrives at the destination.
 - (Optional) Display instructions on how to unlock the robot's cargo compartment to retrieve groceries.

7. Admin Dashboard

Description:

The dashboard interface is shared between employees and managers. Manager accounts have additional privileges, such as access to the financial reports and analytics, while employee accounts are restricted to inventory and order management. Access is restricted to authenticated users with appropriate roles.

Function specifications:

- Requires login with employee or manager credentials to access the dashboard
- Allows adding, editing, or deleting products
- Updates product inventory quantities
- Allows viewing and querying of order records
- Allows managers to view financial metrics such as revenue summaries
- Enforces role-based access, restricting features according to employee or manager privileges

Backend Components

1. Authentication Service

Description:

Handles user authentication and authorization for the OFS system. It verifies user credentials, generates secure access tokens, and enforces role-based access to ensure that only authorized users can access protected resources.

Functional Specifications:

- Receives the username and password from the user attempting to log in
- Validates the credentials against stored user records in the database
- Stores passwords securely using hashing and salting techniques
- Generates a secure access token upon successful authentication
 - Token includes role information for role-based access control
- Verifies tokens on each request for protected resources
- Invalidates tokens after a set expiration period

- Allows users to log out, invalidating the token immediately

2. Product Service

Description:

Manages the product catalog for the OFS system. It handles all interactions with the database related to product information, including retrieving product details for display, updating inventory, and processing admin changes to the catalog.

Function Specifications:

- Retrieves product information from the database (including name, description, price, weight, dietary/organic labels, and stock availability) for frontend display
- Allows authorized users (admins) to create new product records in the database
- Allows authorized users to update existing product details (including price, description, weight, and stock quantity)
- Allows authorized users to remove products from catalog
- Reports current stock to frontend
- Processes filtering and search requests from the frontend by querying the database for matching product records

3. Cart and Order Service

Description:

Manages customer shopping carts and the processing of orders. It calculates the order totals, applies delivery fees based on order weight, validates stock, updates inventory and revenue after purchases and stores completed orders in the database.

Function Specifications:

- Allows adding, updating, and removing of items from a customer's cart
- Calculates the total price and total weight of all items in the cart
- Applies a \$10 delivery fee if the total order weight is more than 20 pounds, otherwise delivery will be free
- Validates stock levels before finalizing the order
- Stores completed order in the database
- Updates product inventory quantities after a successful order
- Updates OFS revenue records after a successful order
- Provides order status updates for frontend display

4. Delivery Scheduling Service

Description:

Manages the assignment of customer orders to OFS's self-driving delivery robots. It groups orders into trips while respecting constraints (such as maximum orders per trip

and total weight) and generates optimized delivery batches to minimize delivery time and ensure efficient use of the delivery robots.

Function Specifications:

- Retrieves all pending orders that are ready for delivery from the Cart & Order Service and/or database
- Groups orders into delivery trips, ensuring no trip exceeds 10 orders or 200 pounds total weight
- Assigns orders to delivery batches based on their destination
- Assigns batches to robots
- Passes delivery addresses to route optimization component for computing routes
- Provides delivery trip status and progress for frontend display or admin dashboard

5. Route Optimization Service

Description:

Calculates the most efficient delivery path for a batch of customer orders assigned to a robot. It considers delivery addresses, distances, and real-time traffic conditions to minimize travel time. Provides optimized routes to the Delivery Scheduling Service, which then assigns them to the delivery robots.

Function Specifications:

- Receives a list of delivery addresses for a batch of orders from the Delivery Scheduling Service
- Validates that all addresses are reachable
- Calculates the most efficient route considering distance and traffic conditions
- Ensures route starts at the robot's current location or home base and ends after final delivery
- Respects delivery constraints
- Formats route so that Delivery Scheduling Service can directly use it for execution
- Handles errors or unreachable addresses by returning an error message to the scheduling service

Database Design Overview

Main tables:

1. Users

- Stores customer, employee, manager information (login credentials and roles)
- Links to orders placed by customers

2. Products

- Stores product catalog information
- Links to inventory and order items

3. Inventory

- Tracks stock levels for each product
- Updated when orders are placed or when admin adjusts quantities

4. Orders

- Stores information about customer orders & status
- Links user who placed order to products purchased

5. Deliveries

- Tracks delivery trips for robots
- Includes which orders are assigned to each trip

6. Robots

- Stores information about robots (maximum capacity, current status, current location)
- Used to assign trips

7. Revenue

- Tracks total revenue
- Linked to orders
- Used by admin dashboard to display metrics

Delivery Scheduling Logic

The Delivery Scheduling Logic is the core algorithm within the Fleet Manager service. Its primary goal is to batch incoming orders efficiently while adhering to the physical constraints of the delivery robots and ensuring timely delivery. It utilizes a "Greedy Best-First" strategy to prioritize orders that are geographically close or nearing their delivery deadline.

Logic Workflow:

Pre-Computation (Fee & Weight Check):

Trigger: Upon checkout, the system calculates the total weight of the cart.

Fee Rule: If the total weight is under 20 lbs, the delivery fee is waived (\$0). If the weight exceeds 20 lbs a \$10 delivery fee is charged.

Route Optimization Logic

The Route Optimization Service acts as the intelligence layer for the robot fleet. It accepts a grouped “batch” of orders from the Scheduler and calculates the most efficient traversal path. This service leverages the Google Maps API to solve the Traveling Salesperson Problem (TSP), ensuring deliveries are made safely and on time despite high traffic density in Downtown San Jose.

Constraints & Inputs:

Batch Size Limit: Max 10 distinct delivery stops per trip.

Payload Limit: Max 200 lbs total weight (validated upstream by Scheduler).

Geographic Bounds: Service is restricted to the Downtown San Jose / SJSU campus area.

Logic Workflow:

- Input Parsing:
Receive a Batch_ID containing a list of N orders (where $1 \leq N \leq 10$).
Extract the destination addresses (lat/long) for all orders in the batch.
Identify the robots current starting position (Fulfillment Center or current location).
- Cost Matrix Generation (Google Maps Integration):
API Call: Query the Google Maps Distance Matrix API.
Request: Send all N+1 coordinates (Start + N destinations).
Response: Receive a matrix of travel times (duration_in_traffic) between every pair of points.
Note: Real-time traffic data is critical here to avoid congestion during peak hours.
- Sequence Optimization (Algorithm):
Objective Function: Minimize Total Trip Duration.
Algorithm: Apply a heuristic TSP solver (e.g., Simulated Annealing or Christofides algorithm) to order the stops.
Result: A sorted array of Order_IDs representing the optimal delivery sequence
- Path Generation & Telemetry:
API Call: Query the Google Maps Directions API for the finalized sequence.
Output: Retrieve the turn-by-turn navigation instructions and the encoded polyline for the entire route.
Dispatch: Transmit the route to the Robot Agent / Onboard Controller and update the Order Tracking service for customers.

Dynamic Re-routing (Edge Case):

If the Robot Telemetry Service detects a delay exceeding a defined threshold (e.g., 10 minutes stuck in traffic), the service re-triggers Step 2 from the robot's current location to find a faster alternative path to the remaining stops.

Hardware/Software List

Hardware:

- Linux or windows server
- Laptops
- Robot delivery vehicles
- End-user devices

Software:

- React (Frontend framework)
- FastAPI (Backend Framework)
- MySQL (Relational Database)
- Apache (Web Server)
- Google Maps API
- JWT Authentication

Tools:

- Git & GitHub
- VS Code
- Postman (API testing)
- MySQL Workbench
- Figma (UI mockups)